

CAMBRIDGE INSTITUTE OF TECHNOLOGY

NORTH CAMPUS, BENGALURU - 562110

Off International Airport Road, Kundana, Bengaluru - 562110

(Approved by AICTE, Affiliated to VTU, Belagavi, Recognized by Govt. of Karnataka, Certified by ISO 9001:2015)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Ref No: CSE/Proj/2026/67

Circular Date: 05.10.2026

Last Date for Submission: 08.10.2026

MINI PROJECT TITLE & ABSTRACT SUBMISSION

Academic Year 2026–2027 | 5th Semester B.E. (Computer Science & Engineering)

1. PROJECT BATCH DETAILS

(Guideline 1: Batch Composition)

Sl. No.	Student Full Name	University Seat Number (USN)
1	Narayana Gayathri	1AJ24CS142
2	VIJAY PURANDARE	1AJ25CS415
3	SONUSHREE S	1AJ24CS132
4	SYED MOHMMED RAIYAN	1AJ25CS415

2. MINI PROJECT TITLE

(Guideline 2: Clear, innovative, relevant to course objectives)

PROPOSED TITLE:

CloudBase IDE: A Container-Isolated, Local-First Browser Development Environment with Storage Guardian Sandboxing and Offline Neural Intelligence

Alternative Formulations (for Discussion with Faculty Coordinator):

- CloudBase:** A Secure, Containerized Web-Based IDE with Zero-Telemetry Offline AI and Strict File System Sandboxing
- Local-First Container IDE:** Hardware-Isolated Multi-Language Runtime and Storage Guardian Defense-in-Depth for Browser Development

3. ABSTRACT

(Guideline 3: Problem statement and process of mini project conduction)

Modern software development relies heavily on either cloud-hosted web IDEs (such as GitHub Codespaces and Replit) or traditional desktop editors (such as VS Code). While cloud IDEs enable universal browser accessibility, they introduce critical vulnerabilities: recurring subscription expenses, unpredictable network latency, vendor lock-in, and the exposure of proprietary intellectual property and source code to remote third-party cloud servers. Conversely, traditional desktop editors execute with uncontained host-level operating system privileges, leading to host environment pollution across conflicting runtime versions and leaving host workstations exposed to malicious dependency scripts (e.g., typosquatting or arbitrary execution via untrusted npm/pip packages).

CloudBase IDE resolves this fundamental architectural dilemma by engineering an enterprise-grade, **local-first, browser-accessible integrated development environment** that guarantees absolute privacy, hardware-enforced container isolation, and zero-compromise developer ergonomics. The system integrates: **(1)** a VS Code-inspired React 18 single-page application incorporating Monaco Editor (v0.52) and an interactive xterm.js canvas terminal communicating over low-latency full-duplex WebSockets; **(2) Storage Guardian**, a defense-in-depth security subsystem enforcing strict canonical path jailing within %LOCALAPPDATA%\CloudBaseIDE\Projects, mitigating Zip-Slip vulnerabilities and rejecting host directory escapes; **(3)** a hardware-isolated container execution runtime managed via Dockerode on WSL2, providing instantly provisioned micro-containers for Python 3.12, Node.js 22, React SPA, Java 21, C++ (GCC 13), and Go 1.22 with strict CPU and RAM ceilings; **(4)** an on-device, 100% offline neural coding assistant leveraging local Qwen2.5-Coder weights via Ollama with typewriter streaming and side-by-side Monaco diff inspection; and **(5)** a native persistence layer powered by Node.js 24's zero-dependency node:sqlite engine running in Write-Ahead Logging (WAL) mode.

The **process of mini project conduction** adheres to a systematic six-phase Agile-Iterative engineering lifecycle: **Phase 1:** Literature survey, threat modeling against path traversal, and SRS formalization; **Phase 2:** Multi-tier architecture design and security perimeter specification; **Phase 3:** Frontend workspace development with Monaco Editor and Storage Guardian canonical path validation; **Phase 4:** Docker container lifecycle orchestration and WebSocket terminal multiplexing; **Phase 5:** Offline LLM token streaming integration, visual diff review modal, and native SQLite schema migrations; and **Phase 6:** Automated security validation (Vitest test suite), PowerShell diagnostic health checks, and academic documentation. Benchmarking verifies sub-50ms editor responsiveness, sub-10ms terminal input/output latency, 100% path traversal immunity, and complete offline autonomy with zero telemetry.

Local-First Architecture

Docker Container Isolation

Storage Guardian

Monaco Editor

xterm.js Terminal

Offline LLM (Qwen2.5-Coder)

Native SQLite WAL

100% Privacy Preserving

4. PROBLEM STATEMENT & PROJECT OBJECTIVES

Problem Statement: Existing cloud-based IDEs compromise developer privacy, mandate continuous high-bandwidth internet connectivity, and incur recurring compute costs, whereas traditional desktop environments run unsandboxed with full host privileges, creating severe dependency conflicts and system vulnerability risks from third-party scripts.

Measurable Project Objectives:

- Construct a high-performance web IDE with Monaco Editor, breadcrumbs, multi-tab editing, and status monitoring.
- Implement **Storage Guardian** to mathematically eliminate directory traversal (`..`), symlink escapes, and host drive attacks.
- Deploy hardware-level containerization for multi-language runtimes (Python, Node.js, React, Java, C++, Go) with resource limits.
- Build an interactive full-duplex terminal using `xterm.js` and `Socket.IO` with character echo and `Ctrl+C` signal termination.
- Integrate on-device neural AI using Ollama for typewriter code generation, diff review, and heuristic offline fallbacks.
- Persist project metadata and audit logs using Node.js 24 native `node:sqlite` in WAL mode with zero compilation hurdles.

5. PROCESS OF MINI PROJECT CONDUCTION

(Implementation Phases & Conduction Roadmap)

Phase 1: Requirements & Threat Modeling (Weeks 1–2)

Surveying container isolation literature, defining threat boundaries against host directory attacks, and formalizing Software Requirement Specifications (SRS).

Phase 2: Architectural Design (Weeks 3–4)

Architecting the multi-tier topology separating presentation, gateway, Docker runtime, and local AI. Designing REST/WebSocket interface protocols.

Phase 3: Frontend & Security Layer (Weeks 5–6)

Implementing the React 18 UI with Monaco Editor 0.52 and authoring the Storage Guardian canonical path validation engine in TypeScript.

Phase 4: Container & Terminal Runtime (Weeks 7–8)

Integrating Dockerode over Docker WSL2 named pipes, writing multi-runtime Dockerfiles, and building the `xterm.js` full-duplex WebSocket terminal.

Phase 5: Offline AI & Native Persistence (Weeks 9–10)

Streaming Ollama tokens from local Qwen2.5-Coder weights, implementing visual Monaco diff inspection, and setting up native SQLite WAL mode.

Phase 6: Testing, QA & Documentation (Weeks 11–12)

Executing Vitest security test suites, automating PowerShell health checks, and compiling final academic reports, manuals, and presentation decks.

6. TECHNICAL STACK & HARDWARE / SOFTWARE SPECIFICATIONS

Layer / Subsystem	Technology Selected	Architectural Rationale
Frontend Studio	React 18, Vite 6, Tailwind CSS	Sub-50ms HMR build times, concurrent rendering, tokenized VS Code Light theme.
Editor Engine	Monaco Editor 0.52	Native VS Code editing core, multi-cursor, syntax tokenization, visual diff viewer.
Terminal Interface	xterm.js 5.5 + Socket.IO 4.8	Canvas-accelerated terminal emulation, ANSI sequences, bidirectional I/O bridge.
Security Sandbox	Storage Guardian Subsystem	Canonical path jailing (%LOCALAPPDATA%), host blacklist, Zip-Slip immunity.
Runtime Engine	Docker Desktop (WSL2) + Dockerode	Hardware-level container isolation, resource caps (CPU/RAM), multi-language images.
Offline Neural AI	Ollama API + Qwen2.5-Coder	100% private, on-device intelligence, typewriter streaming, zero external API costs.
Database Persistence	Native node:sqlite (Node 24)	Built-in WAL mode persistence, eliminates C++ node-gyp build dependencies.

7. RELEVANCE TO 5TH SEM CSE CURRICULUM & INNOVATION

Curriculum Alignment: Directly integrates concepts from *Web Technology* (React 18, WebSockets, RESTful APIs), *Operating Systems* (Process isolation, PTY terminal emulation, signal interrupts), *Cloud Computing & System Security* (Containerization, Linux cgroups/namespaces, directory path traversal defense), and *Database Management Systems* (WAL transactions, relational schemas).

Innovation (Guideline 2 Compliance): Unlike existing desktop editors that grant unrestricted host OS execution privileges, CloudBase IDE guarantees dual-barrier security (Docker isolation + Storage Guardian jailing) while embedding an offline AI coding agent that runs completely independent of internet connectivity.

Student Batch Signatures 1. Narayana G. 2. Vijay P. 3. Sonushree S. 4. Syed Raiyan	Mini Project Coordinator Prof. Bandhavya G. / Prof. Anu Pooja / Prof. Varshini T. R.	Head of Department Dr. Rajshekhar M., HOD CSE, CITNC
---	---	--